

openIMIS Backend (anies_core_mis_be) — Vue d'ensemble complète

Ce document explique l'architecture, la configuration, les composants clés et les étapes d'exécution de ce backend Django modulaire (openIMIS), ainsi que les points d'attention et le dépannage courant (notamment la base de données).

1. Architecture générale

- Application Django modulaire: la base (openIMIS/) charge dynamiquement des modules via `OPENIMIS_APPS`.
- Exposition API: GraphQL via `graphene_django` (schéma `openIMIS.schema.schema`) et REST (DRF) pour certaines surfaces (ex. FHIR R4, Swagger via `drf_spectacular`).
- Websockets: via `channels`.
- Planification de tâches: `django_apscheduler` et `apscheduler_runner`.
- Recherche: `django_opensearch_dsl` vers OpenSearch en option.
- Sécurité et contrôle d'accès: `rules`, `axes`, `middlewares` custom `core.middleware.*`.

Arborescence (extraits utiles):

- `openIMIS/openIMIS/settings/`: paramètres Django split-settings (base, database, logging, etc.).
- `src/`: modules packagés (pip) openIMIS, p.ex. `openimis-be-core`, `openimis-be-claim`, `openimis-be-insuree`, etc.
- `docker-compose.yml`: services db (PostgreSQL), `db-mssql` (MS SQL, option), `opensearch`, `opensearch-dashboards`.
- `data/` et `dbscript/`: SQL, schémas, fixtures utilitaires.
- `script/`: scripts de gestion (entrypoint, requirements, setup dev, etc.).

2. Paramètres Django (split settings)

Les paramètres sont chargés via `openIMIS/openIMIS/settings/__init__.py` qui inclut:

- `security.py`, `base.py`, `database.py`, `logging.py`, `sentry.py`, `scheduler.py`, `queue_cache.py`, `opensearch.py`, puis un fichier selon `MODE` (`DEV.py`/`PROD.py`).
- Chargement optionnel d'un fichier `.env` (défini via `LOAD_ENV`, `.env` par défaut).

Points clés dans `base.py`:

- `INSTALLED_APPS` de base + `OPENIMIS_APPS` (modules), puis `apscheduler_runner`, `signal_binding`, `receiver_binding`.
- `REST_FRAMEWORK`: authentification JWT custom `core.jwt_authentication.JWTAuthentication` + `Basic/Session`. Handler d'exception `openIMIS.ExceptionHandlerDispatcher.dispatcher`.
- `GRAPHENE`: schéma `openIMIS.schema.schema`, `middlewares`, option debug en `DEBUG`.
- `Middleware`s de sécurité et de rate-limit (`Axes`, `CSP`, `headers`).
- Fichiers statiques via `whitenoise`.

3. Configuration base de données

Gérée dans `openIMIS/openIMIS/settings/database.py` via variables d'environnement.

- `DB_DEFAULT` (postgresql|mssql, défaut: postgresql) pilote la branche de config.
- Postgres (défaut):
- `PSQL_DB_ENGINE` (défaut `django.db.backends.postgresql`)
- `PSQL_DB_NAME`, `PSQL_DB_USER`, `PSQL_DB_PASSWORD`, `PSQL_DB_HOST` (défaut `db`), `PSQL_DB_PORT` (défaut `5432`).
- `OPTIONS`: `{'options': '-c search_path=django,public'}`.
- `MSSQL` (option): via pyODBC, options driver 17.
- `NO_DATABASE=True` permet SQLite pour du run sans DB (dev/test minimal).
- Router additionnel `openIMIS.routers.DashboardDatabaseRouter` si DB dashboard.

Important en Docker: n'utilisez pas `127.0.0.1` comme `PSQL_DB_HOST` dans le conteneur backend. Utilisez le nom du service compose, p.ex. `db`.

4. Services Docker

`docker-compose.yml` fournit:

- `db` (PostgreSQL): image `ghcr.io/openimis/openimis-pgsql:<tag>`, ports `5432:5432`, variables `POSTGRES_*`. Healthcheck `pg_isready`.
- `db-mssql` (option): image `ghcr.io/openimis/openimis-mssql`, port `1433`.
- `opensearch + opensearch-dashboards`: single-node, sécurité désactivée (dev), ports `9200/9600` et `5601`.

L'application backend n'est pas définie ici (ce dépôt est focalisé code); vous pouvez l'exécuter en local (venv) ou créer votre propre service dans un compose séparé.

5. Modules openIMIS

Les modules packagés sont dans `src/` (installables via pip). Exemples: `openimis-be-core`, `openimis-be-claim`, `openimis-be-insuree`, `openimis-be-policy`, `openimis-be-medical`, etc. Ils sont référencés par `OPENIMIS_APPS` pour l'auto-chargement. La liste exacte peut être générée depuis `openimis.json` via `script/modules-requirements.py`.

6. API et schéma

- GraphQL: endpoint typique `/api/graphql` (préfix configurable via `SITE_ROOT`), schéma défini dans `openIMIS/schema.py` avec middlewares (tracing, langue utilisateur, JWT).
- REST: `drf_spectacular` expose une documentation Swagger pour FHIR R4.
- Sécurité JWT: prise en charge HS256 par défaut; RS256 possible si des clés RSA sont fournies (voir README — section JWT).

7. Lancement en local (développeurs)

1) Préparer l'environnement Python:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python script/modules-requirements.py openimis.json > modules-requirements.txt
python -m pip install -r modules-requirements.txt
```

```
cp .env.example .env
# Ajuster les variables DB (Postgres conseillé) dans .env
```

2) Démarrer la base de données (via Docker compose dans ce repo):

```
docker compose up -d db
# Optionnel: opensearch et dashboards
docker compose up -d opensearch opensearch-dashboards
```

3) Migrer et lancer le serveur (depuis openIMIS/):

```
cd openIMIS
python manage.py migrate
python manage.py runserver 0.0.0.0:8000
```

4) (Optionnel) Créer un superuser:

```
python manage.py createsuperuser
```

8. Variables d'environnement essentielles

Voir tableau dans README.md. Les plus critiques:

- MODE (DEV/PROD)
- DB_DEFAULT, PSQL_DB_* ou MSSQL_DB_*
- SITE_ROOT, SITE_URL
- DJANGO_SETTINGS_MODULE (par défaut openIMIS.settings)
- SECRET_KEY (production)
- CSRF_TRUSTED_ORIGINS (prod)

9. Dépannage courant

- Connexion DB refusée 127.0.0.1:5432: dans Docker, utilisez PSQL_DB_HOST=db. Vérifiez docker compose ps et healthcheck. Test: pg_isready -h db -p 5432 -U <user>.
- Migrations incohérentes: assurez-vous que la DB est accessible, puis python manage.py migrate.
- OpenSearch: assurez-vous que le service est UP, configurez OPENSEARCH_HOSTS coté dashboards.
- JWT RS256: générez et placez les clés RSA si nécessaire; sinon HS256 par défaut.

10. Tests, outils et scripts

- Tests unitaires des modules (exemple): python manage.py test --keep claim.
- Outils développeurs: création de modules, installation locale/PyPI, extraction de traductions (voir README — Developer tools).
- Scripts utiles dans script/ (génération requirements, entrypoint, etc.).

11. Sécurité et production

- Middlewares de sécurité et en-têtes CSP/HSTS activés en PROD (voir README — Security Headers).
- Journalisation Sentry optionnelle (IS_SENTRY_ENABLED, SENTRY_DSN).
- Cookies sécurisés pour CSRF/JWT en PROD.

12. Points d'intégration

- Canaux (AMQP) via CHANNELS_HOST pour websockets.
- OpenSearch pour indexation/rapports.
- FHIR R4 pour interop santé; Swagger via drf_spectacular.

13. Résumé exécution rapide

1) `docker compose up -d db` 2) Configurer `.env` avec `PSQL_DB_HOST=db, user/pass/DB`. 3) `python -m pip install -r requirements.txt && python -m pip install -r modules-requirements.txt` 4) `cd openIMIS && python manage.py migrate && python manage.py runserver 0.0.0.0:8000`